

Checklist da entrega vs. desafio AdzHub

Revisão do que o desafio pede (guia + página oficial) contra o que está entregue.

Paper (peso 50%, obrigatório)

Requisito	Status
PDF, máx. 3 MB, português	✓ 318 KB
Curto, estilo arXiv/OpenHands, porém simples	✓ 6 páginas, 2 colunas
Tese defendida (não só descrita)	✓ Harness Tri-Camada; §2.3 mapeia as 5 famílias (Tabela 1) e §3.1 defende a composição
Anotações de estudo / decisões / trade-offs	✓ §1, §2.3, §3.1, §3.3
Considera supercérebro + Apps + APIs	✓ §2.2 (por que grafo, por que tempo é camada, por que App ≠ tool, o que os canais mudam) + §3.2
Pontos críticos por tarefa do gestor	✓ §6 é seção própria: 7 tarefas na Tabela 3 + os dois casos graves em prosa
Achado próprio, medido	✓ Quadro 1: entrada de 2,9k → 11,3k tokens contra ~434 de saída num turno de 5 passos
As 11 perguntas do roteiro	✓ mapa em <code>paper/GUIA-DO-PAPER.md</code>
Figura + tabela + referências	✓ Figura 1, Tabela 1, 6 refs
Fontes de estudo registradas	✓ OpenHands, ReAct, CodeAct, RLM (2512.24601) , Zep/Graphiti, Anthropic

Protótipo (peso 25%, recomendado)

Requisito	Status
Chat web estilo Cursor que ilustra a tese	✓
Trace visível do harness (intent → tools → resposta)	✓ hidratação (fase) + tools por camada + resposta
OpenRouter como motor de LLM	✓ <code>web/js/openrouter.js</code>
Campo na UI para colar <code>OPENROUTER_API_KEY</code>	✓ modal Configurar
Key só na sessão do browser, não persiste no servidor	✓ <code>sessionStorage</code> , nunca vai ao backend
Trocar de modelo	✓ dropdown clicável
Simula tools/APIs/Supercérebro com dataset	✓ 7 mocks cruzados (Housewhey)
Deploy público (Railway recomendado)	✓ https://adzhub-harness-production.up.railway.app
UI/UX	✓ layout de três colunas seguindo o design de referência da AdzHub, uma conversa por tarefa (histórico e trace separados), tema claro/escuro em par com escada de contraste, balão azul do gestor à direita, tabelas estilo Ads Manager, resposta digitada ao vivo, trace agrupado (Nx)
Entrada do composer	✓ + abre menu de tipo num clique e o explorador direto no duplo clique; anexo de CSV/TSV/JSON/TXT/MD lido no browser e cortado em 40 mil caracteres pelo harness; ditado contínuo pela API de fala do navegador, que entra no texto já escrito e só para no clique (sem segunda chave, ver §5 do paper). <code>build/test_voz.js</code> trava as duas regressões
Persona própria (não é chatbot genérico)	✓ NEXO , estrategista de performance, em <code>web/js/nexo.js</code> (fora do runtime)
Consumo de tokens visível pro gestor	✓ por chamada (trace), por turno (rodapé com tabela) e por sessão (medidor no topo), com custo em US\$ quando o provedor devolve
Bônus: roda sem key (modo simulado)	✓ avaliador testa sem colar chave

Formulário

Campo	Valor
Nome completo	Rafael Yran Azevedo
WhatsApp	o mesmo da candidatura
Tipo de harness	Outra / híbrida / própria (Harness Tri-Camada)
Paper (PDF)	<code>paper/paper.pdf</code>
Repositório GitHub (público)	https://github.com/haestare/adzhub-harness
URL da demo	https://adzhub-harness-production.up.railway.app
Notas	só-leitura por design, dados mock, 5 problemas plantados não rotulados, modo simulado + LLM/OpenRouter

Estado do protótipo (o que o avaliador encontra)

Recurso	Situação
Persona própria (NEXO), fora do runtime	✔ <code>web/js/nexo.js</code>
Trace do harness com badge por camada e agrupamento Nx	✔
Medidor de tokens (por chamada, por turno, por sessão)	✔ medido no LLM, estimado com $\approx$ no simulado
Streaming SSE real, com <code>tool_calls</code> remontados	✔
Tabelas estilo Ads Manager + títulos h1-h3	✔ <code>build/test_md.js</code> trava a regressão
Tema claro/escuro persistente	✔
Comandos de barra (10, com paleta)	✔ <code>/tools</code> e <code>/uso</code> deixam o harness inspecionável
Dropdown de modelos	✔ lista viva da API em 3 grupos: recomendados ranqueados (#1 = mais forte, um por família), grátis (custo zero) e o catálogo inteiro com tool-calling, mais "Outro (digitar id)". Fallback local sem rede, que não anuncia nada como grátis. <code>build/test_modelos.js</code> trava a regressão
Deploy automático a cada commit	✔ <code>.githubhooks/post-commit</code> + hook <code>Stop</code> (espelham e empurram o repo de deploy)
Publicação no host	⚠ o Railway enfileira : um build slot por vez, então uma rajada de commits deixa a demo alguns commits atrás enquanto a fila drena. Diagnóstico é a aba Deployments , e a linha embaixo do deployment diz o motivo: <code>Waiting for build slot</code> é fila normal, <code>Deployment queued due to upstream GitHub issues</code> é a integração do Railway travada (problema do host, não nosso), <code>Failed</code> é problema nosso. Redeploy durante a fila piora ; o que acelera é cancelar os enfileirados intermediários. Todo commit no repo vira um build, mesmo fora de <code>web/</code> , então commitar em lote é operacional. Caminho alternativo pronto: <code>.github/workflows/pages.yml</code> , faltando só ligar em Settings → Pages → Source: GitHub Actions (clique humano, a Action não liga sozinha).
Como saber, em 5s, qual versão está no ar	✔ chip <code>build=<data></code> no painel do harness, e <code>GET /js/modelos.js</code> (404 = host num commit antigo)

Melhorias já aplicadas nesta revisão

- Paper: citação do RLM (arXiv:2512.24601) e nod ao GraphRAG, que a página lista.
- Protótipo: o agente agora é instruído a usar **tabela** para dados comparativos (não lista aninhada).

- Protótipo: **tema claro/escuro** com toggle (persistente, sem flash ao abrir).
- Protótipo: dropdown de modelo, tabelas estilo Ads Manager, trace agrupado (`Nx`).
- Protótipo: **streaming SSE real** no modo LLM. O texto aparece token a token enquanto o modelo gera; os `tool_calls` chegam fatiados e são remontados no transporte, então o loop inteiro é streamado sem chamada extra. No modo simulado continua o typewriter (não há LLM para streamar).
- Protótipo: **persona NEXO**. O agente deixou de ser "assistente" e virou estrategista: conclusão primeiro, fato separado de hipótese e de recomendação, opinião amarrada ao número, e discrepância entre Meta e CRM exposta em vez de resolvida em silêncio. A persona vive em `web/js/nexo.js`, separada do runtime, e o modo simulado segue a mesma régua (senão trocar de motor trocaria de agente).
- Protótipo: **medidor de tokens**, por chamada, por turno e por sessão, com custo em US\$ quando o provedor devolve.  O `usage` do streaming chega num chunk com `choices: []`, então o transporte captura antes de descartar o chunk por falta de delta; sem isso o número sumiria sem erro nenhum. `build/test_uso.js` cobre isso e mais 19 asserções, sem rede e sem chave.
- Docs: `paper/GUIA-D0-PAPER.pdf` e `CHECKLIST.pdf` (versões em PDF, mais fáceis de ler).

Pendente no paper (decisão do Rafa, 19/08)

Três assuntos ficaram de fora por escolha e estão descritos em `paper/GUIA-D0-PAPER.md`. Cada um vale meia página no critério "aprofundamento no estudo":

Assunto	Por que renderia ponto
Gestão de contexto em sessão longa	é a consequência direta do Quadro 1; hoje o §7 só admite a lacuna
Como saber se o harness funciona (traces de referência, regressão)	é o assunto que mais sinaliza vaga fundacional
Falha de tool e paralelismo (erro, rate limit, dado parcial)	hoje o paper não fala de erro em lugar nenhum

Opcionais que ficam de fora (não bloqueiam a entrega)

- Ouvir o **podcast AdzHub · Harness** e o **How I AI** e citar uma frase no §1 (reforça o critério "aprofundamento no estudo"). Deixei de fora do paper para não afirmar o que você ainda não ouviu.
- Um segundo cliente no mock, para mostrar multi-conta (o paper já assume um cliente por escolha de MVP).